

Mastermind — Tests

Project 2, Phase 1 · tests/assignment2/MastermindTests.java · EID: ak54582

How the tests run

The suite has no dependencies (no JUnit), so it runs anywhere a JDK is installed. It exits with status 0 only if every check passes.

```
javac -d out src/assignment2/*.java tests/assignment2/*.java
java -cp out assignment2.MastermindTests
```

Tests work at three levels: **pure scoring** (`MastermindScorer.score`), **game API** (`MastermindGame` with a `FixedCodeGenerator`, so the secret is known), and **full console sessions**. For the last level, a scripted input string is fed through `ConsoleRunner` via an injected `BufferedReader/PrintStream`, and the captured output is checked. Test mode's fixed secret makes every case deterministic.

Latest run: 80 passed, 0 failed. The 14 tests below are the most important ones. The table at the end lists all 80 checks by group.

Test cases

T1 Guess repeats a color the secret has only once

[Duplicate colors](#)

Setup	Secret RGBY
Input	Guess RRRR
Expected	1B_0W. The one R in the secret is matched in place and can't also produce whites for the other three Rs.
Why	Catches the most common bug: counting a white for every guess peg whose color appears anywhere in the secret, which would give 1B_3W.
Result	PASS

T2 Guess has more copies of a color than the secret

[Duplicate colors](#)

Setup	Secret BBGO
Input	Guess GBBB
Expected	1B_2W. Position 2 is an exact B. Leftovers: secret {B, G, O}, guess {G, B, B}, which give one white for B (only one B left in the secret) and one for G.
Why	Checks that whites use $\min(\text{secret count}, \text{guess count})$ per color, after exact matches are removed.
Result	PASS

T3 Exact matches take precedence over whites

Duplicate colors

Setup	Secret B000; also secret RRGG
Input	Guess 00BB; guess RGRG
Expected	1B_2W and 2B_2W
Why	If a peg is used as a white before it can be counted as an exact match, the scores come out wrong. These two cases would expose that ordering bug.
Result	PASS

T4 Every secret × every guess matches an independent reference scorer

Scoring (exhaustive)

Setup	All 1,296 default codes (6 colors, 4 pegs), and all 1,024 codes with 4 colors and 5 pegs (many repeats). The reference uses the classic "used-flags" nested-loop algorithm, which is written differently from the counting-map implementation.
Input	1,679,616 + 1,048,576 (secret, guess) pairs; plus 200,000 random pairs with 1–12 pegs and 10 colors
Expected	0 mismatches. Also: <code>score(s,g) == score(g,s)</code> ; black + white ≤ pegs; black == pegs exactly when guess == secret.
Why	Covers every duplicate-color pattern for the default configuration instead of relying on a few chosen examples.
Result	PASS

T5 Invalid guesses are rejected and never consume an attempt (API)

Invalid input

Setup	Default config, secret BGOP
Input	"", BGO (short), BGOPR (long), BGOX (illegal color), 1234, B G O, BGO!, tab, BGOP BGOP
Expected	All 9 throw <code>IllegalArgumentException</code> from <code>submit</code> ; <code>attemptsUsed() == 0</code> and history is empty. Lowercase ybgo is then accepted and stored as YBGO, using exactly one attempt.
Result	PASS

T6 Invalid guesses in a console session

Invalid input

Setup	Default config, secret BGOP
Input	XXXX, BG, BGOPY, empty line, BGOP, N
Expected	Exactly 4 <code>INVALID_GUESS</code> lines. The prompt "Guess 1 of 12" appears 5 times, because the counter does not advance. The game then ends in a win.
Result	PASS

T7 HISTORY command

History

Setup	Default config, secret BGOP
Input	HISTORY, RRRR, YYYY, HISTORY, history, BGOP, N
Expected	The first HISTORY prints "No guesses yet." Later ones list 1. RRRR 0B_0W before 2. YYYY 0B_0W, oldest first. The lowercase command also works, giving 2 history blocks. The next prompt is "Guess 3 of 12", so HISTORY costs no attempt.
Result	PASS

T8 History is read-only and resets between rounds

History

Setup	Fixed secrets BGOP then YYYY
Input	API: history().add(...). Console: BGOP, Y, RRRR, HISTORY, YYYY, N
Expected	add throws UnsupportedOperationException. In round 2, HISTORY shows only 1. RRRR, not round 1's BGOP. Two wins are reported.
Result	PASS

T9 Win, including on the last allowed guess

Win

Setup	Default config (12 guesses), secret BGOP
Input	(a) BGOP immediately. (b) 11 × RRRR, then BGOP.
Expected	(a) Status WON, isSolved() true, "You win!"; any further submit throws IllegalStateException. (b) Status WON, not LOST: the solved check runs before the attempt-limit check.
Result	PASS

T10 Loss after the attempt limit

Loss

Setup	Default config, secret BGOP
Input	12 × RRRR, N
Expected	After 11 wrong guesses, still IN_PROGRESS. After the 12th: LOST, attemptsRemaining() == 0, output contains "You lose!" and "The secret was BGOP."
Result	PASS

T11 6 pegs, 10 digit colors, 3 guesses

Changed configuration

Setup new GameConfiguration(3, 6, "0123456789"), secret 012345, test mode. Same effect as -Dmastermind.guesses=3 -Dmastermind.pegs=6 -Dmastermind.colors=0123456789.

Input (a) 543210, 000000, 012345. (b) New game: 0123, 01234567, ABCDEF, 111111, 222222, 333333.

Expected (a) Banner says "6 pegs"; 0B_6W, 1B_0W; "You win! Solved in 3 guess(es)." (b) The three wrong-shape guesses are all INVALID_GUESS (the 4-peg default is no longer accepted), and the game is lost after exactly 3 valid guesses.

Why Shows that no game logic hard-codes 4 pegs, 12 guesses or BGOPRY.

Result PASS

T12 Edge configurations and configuration validation

Changed configuration

Setup (a) 2 guesses, 1 peg, colors XY, secret Y. (b) Colors abcd, secret abcd. (c) Constructor with bad values.

Input (a) X, Y. (b) ABCD. (c) 11 colors, duplicate color BGOPRB, empty colors, 0 pegs, 0 or negative guesses, a space as a color.

Expected (a) Win on the 2nd guess. (b) Uppercase input maps onto a lowercase color set, so it's a win. (c) Every bad configuration throws IllegalArgumentException; exactly 10 colors and a 1/1/1 configuration are accepted.

Result PASS

T13 Test mode uses fixed secrets in order (first one not skipped)

Regression

Setup Test mode, fixed secrets BGOP, YYYY

Input BGOP, Y, YYYY, N

Expected "[TEST MODE] Secret: BGOP" is printed before "Secret: YYYY", and both rounds are won. This guards against the game starting a round in its constructor and the runner starting another one, which would silently skip the first secret.

Result PASS

T14 Fuzz, end of input, and a full solver run

Robustness

Setup Seeded random input; default and changed configurations

Input 100,000 random strings through validate/submit. 60,000 random console lines (including HISTORY, Y, N, and non-ASCII text such as emoji and CJK). A 1,000,000-character line. Input ending mid-game or at the replay prompt. CRLF line endings. A consistent-guess solver played against all 1,296 default secrets.

Expected validate()==null exactly when submit succeeds, and attempts change only on success. No exception ever reaches the user. The huge line is rejected and play continues. The solver wins all 1,296 games within 12 guesses (measured: worst 9 guesses on RYRP, average 5.765).

Result PASS

All checks by group

Group (method in MastermindTests)	What it covers	Checks	Result
scoringKnownCases	Hand-verified feedback, mostly duplicate colors (T1–T3)	13	13/13
scoringExhaustiveVsOracle	Every pair vs. a reference scorer (T4)	2	2/2
scoringProperties	Symmetry, bounds, perfect-score property, 200k random cases (T4)	4	4/4
configuration	Defaults and invalid settings (T12c)	10	10/10
gameRules	Invalid input, win, loss, last-guess win, history immutability, reset, generators (T5, T8–T10)	15	15/15
consoleSessions	Scripted sessions: test mode, invalid input, HISTORY, win/loss, replay, end of input (T6–T8, T13)	21	21/21
changedConfiguration	6 pegs / digits / 3 guesses, 1 peg, lowercase color set (T11, T12)	9	9/9
fuzz	Random and oversized input (T14)	4	4/4
solverEveryDefaultSecret	All 1,296 secrets played to a win (T14)	2	2/2
Total		80	80/80